



KHOA CÔNG NGHỆ THÔNG TIN

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐẠI HỌC QUỐC GIA HÀ NỘI

**Xây dựng đường cong Elliptic bậc siêu cao
và ứng dụng Chữ ký số**

KHÓA LUẬN TỐT NGHIỆP
NĂM HỌC 2025–2026

Người thực hiện: Trần Mạnh Duy

Giảng viên hướng dẫn: GS. TS. Lê Phê Đô

Nội dung trình bày

01

**GIỚI
THIỆU**

02

**CƠ SỞ
LÝ THUYẾT**

03

**CÀI ĐẶT
HỆ THỐNG**

04

**KẾT
LUẬN**

ECC là nền tảng bảo mật hạ tầng số

TLS 1.3, SSH, Bitcoin (secp256k1), Ethereum (BLS12-381), chữ ký số điện tử

Thực trạng:

- ▶ Hầu hết hệ thống **tái sử dụng** đường cong sinh sẵn bởi NIST, SECG
- ▶ Tham số dùng “hạt giống” không giải thích được $\Rightarrow$ nguy cơ backdoor
- ▶ Sự kiện Dual_EC_DRBG: NSA bị cáo buộc cài cửa hậu vào PRNG chuẩn NIST
- ▶ Thư viện mật mã chủ yếu viết bằng C/C++ $\Rightarrow$ lỗi bộ nhớ nghiêm trọng (Heartbleed 2014, 70% lỗi hổng Microsoft từ memory safety)

Đường cong	$ p $	Bảo mật	NTT	Ngôn ngữ
secp256k1	256 bit	128-bit	Không	C (libsecp256k1)
P-256 (NIST)	256 bit	128-bit	Không	C (OpenSSL)
BLS12-381	381 bit	128-bit	$\nu_2 = 32$	C/Rust (blst)
Curve1024	1024 bit	256-bit	$\nu_2 = 36$	Rust

Hạn chế của các hệ thống hiện tại:

- ▶ Mức bảo mật cố định 128-bit — không đủ cho kịch bản dài hạn
- ▶ Tham số không minh bạch (NIST P-256: seed không giải thích)
- ▶ Phụ thuộc thư viện C/C++ với rủi ro bộ nhớ

3 vấn đề cần giải quyết

1. Tự chủ mật mã

Sinh đường cong bằng quy trình minh bạch, không seed bí ẩn

2. Trần bảo mật

ECC 256-bit $\rightarrow$ 128-bit

Cần 1024-bit $\rightarrow$ **256-bit**

3. An toàn bộ nhớ

Loại bỏ lỗi buffer overflow bằng ngôn ngữ Rust

Yêu cầu kỹ thuật

- ▶ Đường cong pairing-friendly ($k = 18$, hỗ trợ BLS)
- ▶ Cấu trúc NTT-friendly ($\nu_2(p - 1), \nu_2(r - 1) \geq 32$)
- ▶ Tham số có thể kiểm chứng (từ seed T công khai)
- ▶ Chữ ký số hoạt động (Schnorr, ECDSA)

Toán học & Bảo mật

- ▶ Tự chủ sinh đường cong KSS18 trên trường 1024-bit
- ▶ Bảo mật **256-bit** đối xứng (vượt NIST 128-bit)
- ▶ Kháng tấn công TNFS, MOV, Anomalous, Invalid Curve
- ▶ Cấu trúc NTT-friendly (hỗ trợ ZK proof tương lai)

Kỹ thuật & Ứng dụng

- ▶ Ngôn ngữ **Rust** an toàn bộ nhớ tuyệt đối
- ▶ Không phụ thuộc thư viện mã bên ngoài
- ▶ Chữ ký số: **Schnorr, ECDSA** + lý thuyết BLS
- ▶ Bộ kiểm thử toàn diện: unit + integration + attack simulation

Cơ sở toán học: Trường số nguyên tố $\mathbb{F}_p$

Định nghĩa

$\mathbb{F}_p = \{0, 1, \dots, p-1\}$ với phép cộng và nhân $(\text{mod } p)$,
mọi phần tử $a \neq 0$ có nghịch đảo $a^{-1} = a^{p-2} \pmod{p}$

Kỹ thuật then chốt: Phép nhân Montgomery

- ▶ Chọn $R = 2^{1024}$, biểu diễn $\hat{a} = a \cdot R \pmod{p}$
- ▶ Nhân trong không gian Montgomery:
 $\hat{a} \cdot \hat{b} \cdot R^{-1} \pmod{p}$
- ▶ Thay phép **chia cho p** bằng **shift phải**

	Thông thường	Montgomery
Phép chia	$\div p$	shift
Chi phí	$O(n^2) + \text{chia}$	$O(n^2)$

Short Weierstrass trên $\mathbb{F}_p$

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

Phép cộng điểm $P_1 + P_2 = P_3$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2$$

Phép nhân vô hướng:

$[k]P$ bằng thuật toán *Double-and-Add*,
độ phức tạp $O(\log k)$

Vai trò trong ECC

- ▶ Sinh khóa công khai: $Q = [d]G$
- ▶ Cam kết ký: $R = [k]G$
- ▶ Xác minh Schnorr: $[s]G \stackrel{?}{=} R + [e]Q$

Phương pháp Nhân phức (Complex Multiplication)

Ý tưởng cốt lõi

Thay vì chọn (a, b) ngẫu nhiên rồi đếm điểm,
CM *đảo chiều*: bắt đầu từ bậc nhóm r mong muốn $\Rightarrow$ suy ra đường cong

Điều kiện CM: Tồn tại đường cong trên $\mathbb{F}_p$ với vết Frobenius t khi:

$$4p = t^2 + |D| \cdot y^2 \quad (D < 0 \text{ là biệt thức CM})$$

Quy trình 4 bước:

1. Tính j -invariant từ đa thức lớp Hilbert $H_D(x) \pmod{p}$
2. Suy ra hệ số đường cong (a, b) từ j
3. Kiểm tra xoắn (twist test): chọn đường cong có $r \mid \#E$
4. Tìm điểm sinh G bậc r bằng nhân cofactor

02 | Cơ sở lý thuyết toán học

Thuật toán Cocks-Pinch: Truyền thông & Cải tiến

Điều kiện: $r = \Phi_{18}(T) = T^6 - T^3 + 1$, $4p = t^2 + 3y^2$

4 bước truyền thông:

1. Sinh r từ T ngẫu nhiên, kiểm tra tính nguyên tố
2. Tính $\beta = \sqrt{-3} \pmod{r}$ (Tonelli-Shanks)
3. $t_0 \equiv T^i + 1$, $y_0 \equiv (t_0 - 2)/\beta \pmod{r}$
4. Quét lưới $(h_t, h_y) \in [-20, 20]^2$: $p = (t^2 + 3y^2)/4$ nguyên tố?

Cải tiến 1: NTT-friendly r

$$r - 1 = T^3(T^3 - 1)$$

Nếu $T \equiv 0 \pmod{2^{12}}$:

$$\nu_2(r - 1) = 36 \text{ (đảm bảo)}$$

Không cần rejection sampling

Cải tiến 2: NTT-friendly p

Mở rộng lưới nâng:

$$\text{range} = 20 + 2^{\max(0, s_p - 36)}$$

$$\text{Kiểm soát } \nu_2(p - 1) \geq 36$$

Vượt BL S12.381 ($\nu_2 = 32$)

Schnorr

Ký: $k \xleftarrow{R} [1, r-1]$, $R = [k]G$

$e = H(R_x \| R_y \| m) \bmod r$

$s = k + e \cdot d \pmod{r}$

Chữ ký: $\sigma = (R, s)$ — 384 byte

Xác minh: $[s]G \stackrel{?}{=} R + [e]Q$

ECDSA (FIPS 186-4)

Ký: $k \xleftarrow{R} [1, r-1]$, $R = [k]G$

$r_{\text{sig}} = R_x \bmod n$, $e = H(m)$

$s = k^{-1}(e + r_{\text{sig}} \cdot d) \bmod n$

Chữ ký: (r_{sig}, s) — 256 byte

Xác minh: $[u_1]G + [u_2]Q$, kiểm P_x

	Schnorr	ECDSA
Kích thước chữ ký	384 byte	256 byte
Chứng minh bảo mật	RO Model + DL	Heuristic
Hỗ trợ tổng hợp	Có (MuSig)	Không
Chuẩn	ISO 14888-3	FIPS 186-4

Phép ghép cặp song tuyến (Bilinear Pairing)

$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ thỏa mãn:

$$e([a]P, [b]Q) = e(P, Q)^{ab} \quad (\text{song tuyến})$$

Quy trình BLS:

- ▶ Ký: $\sigma_i = [d_i] H(m)$
- ▶ Tổng hợp: $\sigma = \sigma_1 + \dots + \sigma_n$
- ▶ Xác minh:
 $e(\sigma, G_2) \stackrel{?}{=} e(H(m), \sum Q_i)$

Vai trò đường cong KSS18:

- ▶ $\mathbb{G}_1 \subset E(\mathbb{F}_p)$
- ▶ $\mathbb{G}_T \subset \mathbb{F}_{p^{18}}^*$
(trường 18432-bit)
- ▶ Optimal Ate pairing:
vòng lặp Miller $O(\log r)$

Cài đặt đầy đủ phép ghép cặp: hướng phát triển tương lai

Ngăn xếp phần mềm

- 5 Schnorr / ECDSA
- 4 KeyPair + Hash mở rộng
- 3 AffinePoint — nhóm EC
- 2 PrimeField — Montgomery
- 1 U1024 — bignum 1024-bit

Đặc điểm thiết kế

- ▶ **Rust** — ownership + borrow checker loại bỏ lỗi bộ nhớ tại compile-time
- ▶ **Zero dependency** — không dùng thư viện mật mã bên ngoài
- ▶ **Zero-cost abstraction** — trait PrimeFieldConfig với hằng số compile-time, inline hoàn toàn
- ▶ **Tham số từ file TOML** — thay đổi đường cong không cần biên dịch lại toàn bộ

Thuật toán Cocks-Pinch cải tiến với ràng buộc NTT:

Tham số	CP Truyền thống	CP Cải tiến
Ràng buộc T	Không	$T \equiv 0 \pmod{2^{12}}$
$\nu_2(r-1)$	Ngẫu nhiên (1-3)	≥ 36 (đảm bảo)
$\nu_2(p-1)$	Ngẫu nhiên (1-3)	≥ 36 (lưới mở rộng)
Lưới (h_t, h_y)	$[-20, 20]^2$	$[-21, 21]^2$
Số lần thử	100-500	3.000-6.000
Thời gian	2-10 s	30-60 s
So với BLS12-381 ($\nu_2 = 32$)	Không đạt	Vượt

Nhận xét: Chi phí sinh tham số tăng $\sim 10\times$ nhưng chỉ chạy **một lần** — đổi lại cấu trúc NTT vĩnh viễn cho mọi phép tính trên đường cong.

03 | Cài đặt hệ thống



Tham số đường cong Curve1024

Phương trình: $E : y^2 = x^3 + 5$ trên $\mathbb{F}_p$

$k = 18$, $D = -3$, $|p| = 1024$ bit, $|r| = 512$ bit

Bậc nhóm r — biểu diễn 64-bit limb (MSB $\rightarrow$ LSB):

limb 15-8: 0000000000000000 $\times 8$ limbs (50% = 0)

limb 7: 8e09c0b6aa3a6250

limb 6: 1809d08152ef6657

limb 5: e905d9793d7814b9

limb 4: 5315dd1940ee9641

limb 3: e67861624813833970

limb 2: b962e6f67bcf6422

limb 1: afc3f5b9ba363faf

limb 0: 64eb2200000001

Đặc trưng cấu trúc:

- 8/16 limb bằng 0 $\Rightarrow$ dễ nhận diện, kiểm tra khi debug

03 | Cài đặt hệ thống

So sánh hiệu năng



Đường cong	$ p $	Ký	Xác minh	Bảo mật
secp256k1	256 bit	~0.2 ms	~0.4 ms	128-bit
P-384	384 bit	~1 ms	~2 ms	192-bit
BLS12-381	381 bit	~1.5 ms	~3 ms	128-bit
Curve1024	1024 bit	1.55 s	2.83 s	256-bit

Bảng: *

Schnorr sign/verify. Curve1024: Criterion, release build, LLVM O3, không AVX-512

Kịch bản phù hợp:

- ▶ Ký tài liệu pháp lý, ký phần mềm khi phát hành
- ▶ Chứng chỉ PKI, giao dịch tài chính giá trị lớn
- ▶ Dấu tay số, chữ ký số, chữ ký số, chữ ký số

Kết quả

- ✓ Tự chủ sinh đường cong KSS18 1024-bit, không hàng số nước ngoài
- ✓ Bảo mật 256-bit, vượt NIST 2030+
- ✓ Rust — an toàn bộ nhớ tuyệt đối
- ✓ Schnorr & ECDSA hoạt động đúng

Đánh giá an toàn

Pollard- ρ	$O(2^{256})$ — an toàn
MOV	18432-bit — an toàn
Anomalous	$\#E \neq p$ — miễn nhiễm

Hạn chế

- ▶ **Timing side-channel**
Double-and-Add rò rỉ bit scalar
⇒ cần Montgomery Ladder
- ▶ **Tọa độ Affine**
1 nghịch đảo/phép cộng điểm
⇒ cần Jacobian/Projective
- ▶ **Hiệu năng**
~8000× chậm hơn secp256k1
(do trường lớn + chưa tối ưu ASM)

Ngắn hạn — Hiệu năng

- ▶ **Tọa độ Jacobian/Projective**
1 nghịch đảo/lần nhân vô hướng
thay vì 1024 $\Rightarrow$ dự kiến $\sim 10\times$
- ▶ **Montgomery Ladder**
Constant-time, loại bỏ
timing side-channel
- ▶ **NAF/Sliding Window**
Giảm $\sim 25\%$ số phép add

Trung & Dài hạn

- ▶ **Phép ghép cặp đầy đủ**
Vòng lặp Miller + final exp
trên $\mathbb{F}_{p^{18}}$
 $\Rightarrow$ kích hoạt BLS signatures
- ▶ **MSM Pippenger**
Tối ưu xác minh đa chữ ký
- ▶ **Giao thức ngưỡng**
MuSig2/FROST (t -trong- n)
- ▶ **Kiểm định hình thức**
Creusot hoặc Kani cho Rust

Cảm ơn thầy cô đã lắng nghe!